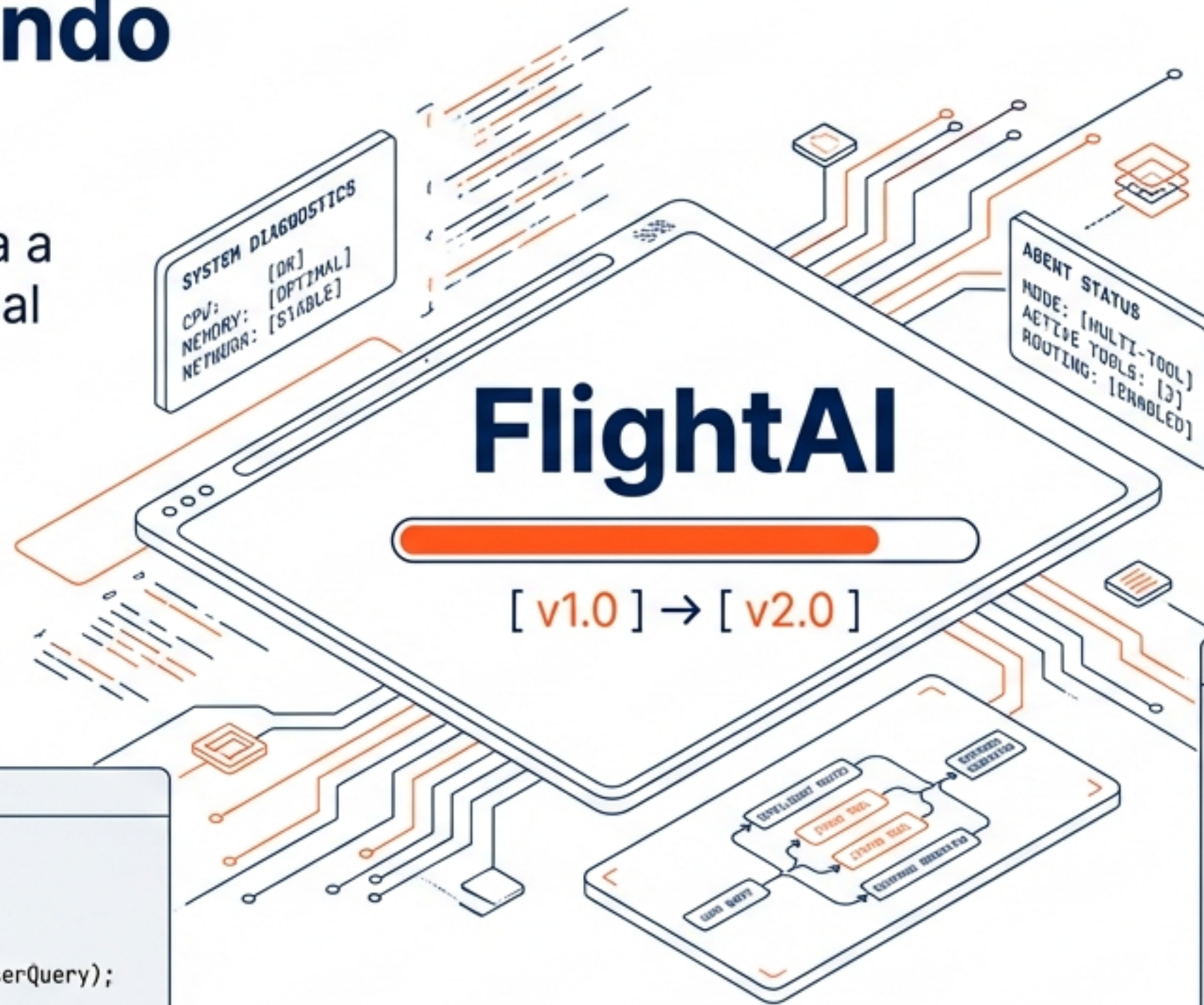


Práctica 02: Evolucionando FlightAI

De Herramienta Única a
Agente Multi-Funcional



Misión: Actualizar el código base del asistente para soportar múltiples herramientas simultáneamente.

Estado Actual: Capacidad limitada a consultas de precios.

Objetivo: Integrar el módulo 'Consulta de Estado de Vuelos' y desplegar la lógica de enrutamiento inteligente.

LEGACY CODE (v1.0)

```
// single_tool_agent.js
function processQuery(userQuery) {
  // Only price check supported
  if (userQuery.includes("price")) {
    return flightPriceTool.getPrice(userQuery);
  } else {
    return "Sorry, I can only check prices.";
  }
}
```

EVOLVED AGENT (v2.0)

```
// multi_tool_agent.js
import { getPrice, getStatus } from './tools';

async function processQuery(userQuery) {
  // Intelligent routing logic
  const intent = await classifyIntent(userQuery);
  switch (intent) {
    case 'PRICE_CHECK':
      return await getPrice(userQuery);
    case 'STATUS_CHECK':
      return await getStatus(userQuery);
    default:
      return "How can I assist with your flight?";
  }
}
```


El Problema de la Herramienta Única



Situación

Actualmente, FlightAI maneja consultas de precios a la perfección.

El Conflicto

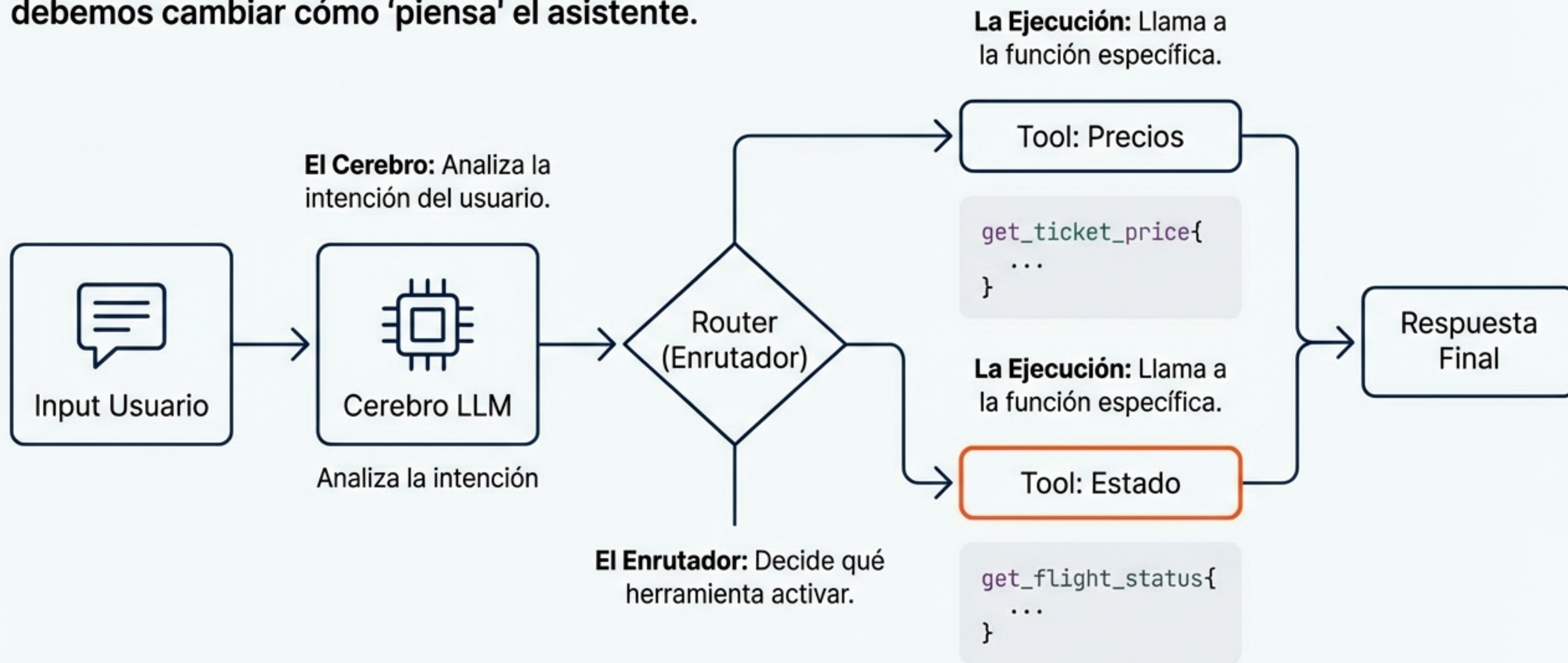
Los clientes están frustrados. En el mundo real, necesitan saber si su vuelo saldrá a tiempo, no solo cuánto cuesta.

El Desafío Técnico

El sistema actual asume que toda entrada es una solicitud de precio. Necesitamos romper esa limitación y enseñar al asistente a discernir entre intenciones.

Arquitectura del Sistema Multi-Agente

Para resolver el problema, no basta con añadir código; **debemos cambiar cómo 'piensa' el asistente.**



Paso 1: La Capa de Datos

Antes de poder consultar, necesitamos datos.

Implementaremos una base de datos simulada justo debajo del diccionario `existing`.

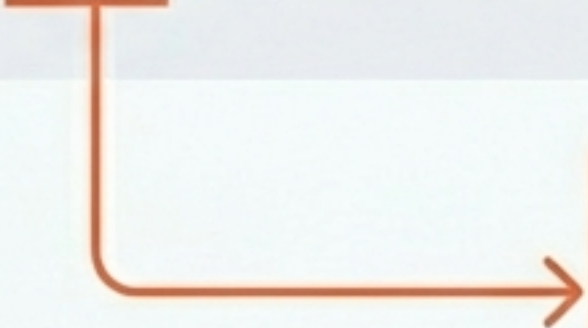
Esta estructura clave-valor permite una recuperación rápida basada en el número de vuelo.

```
# Base de datos simulada de estados de vuelos
flight_status_db = {
    'FA101': 'On Time',
    'FA202': 'Delayed 2 hours',
    'FA303': 'Cancelled',
    'FA404': 'Boarding'
}
```


Paso 2: La Lógica de Recuperación

Creamos el puente entre la IA y los datos. La función `get_flight_status` recibe un número de vuelo y consulta la base de datos.

```
def get_flight_status(flight_number):  
    return flight_status_db.get(flight_number, 'Status Unknown')
```

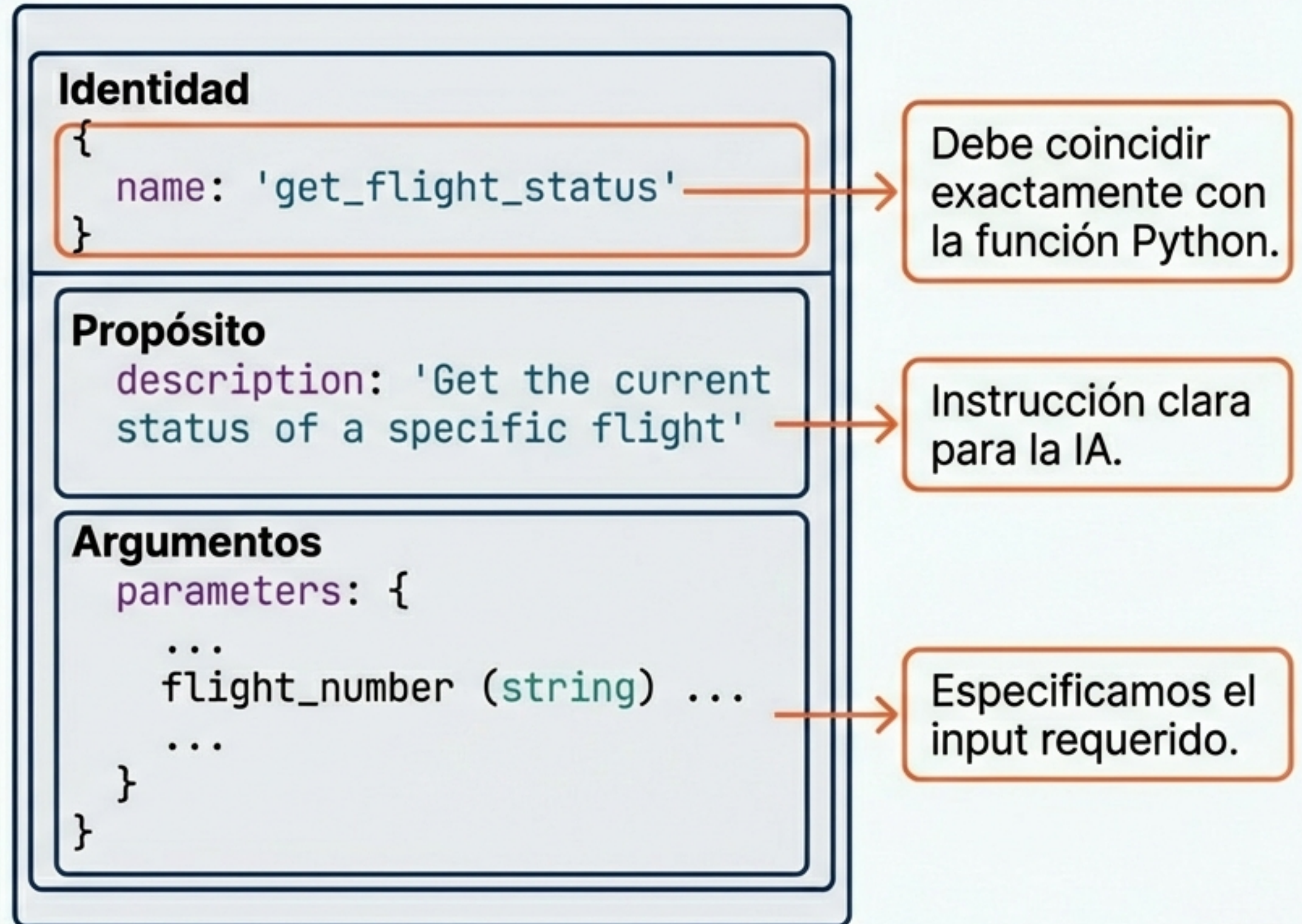


Consejo Técnico: Usamos `.get()` para manejar claves inexistentes sin romper el programa.

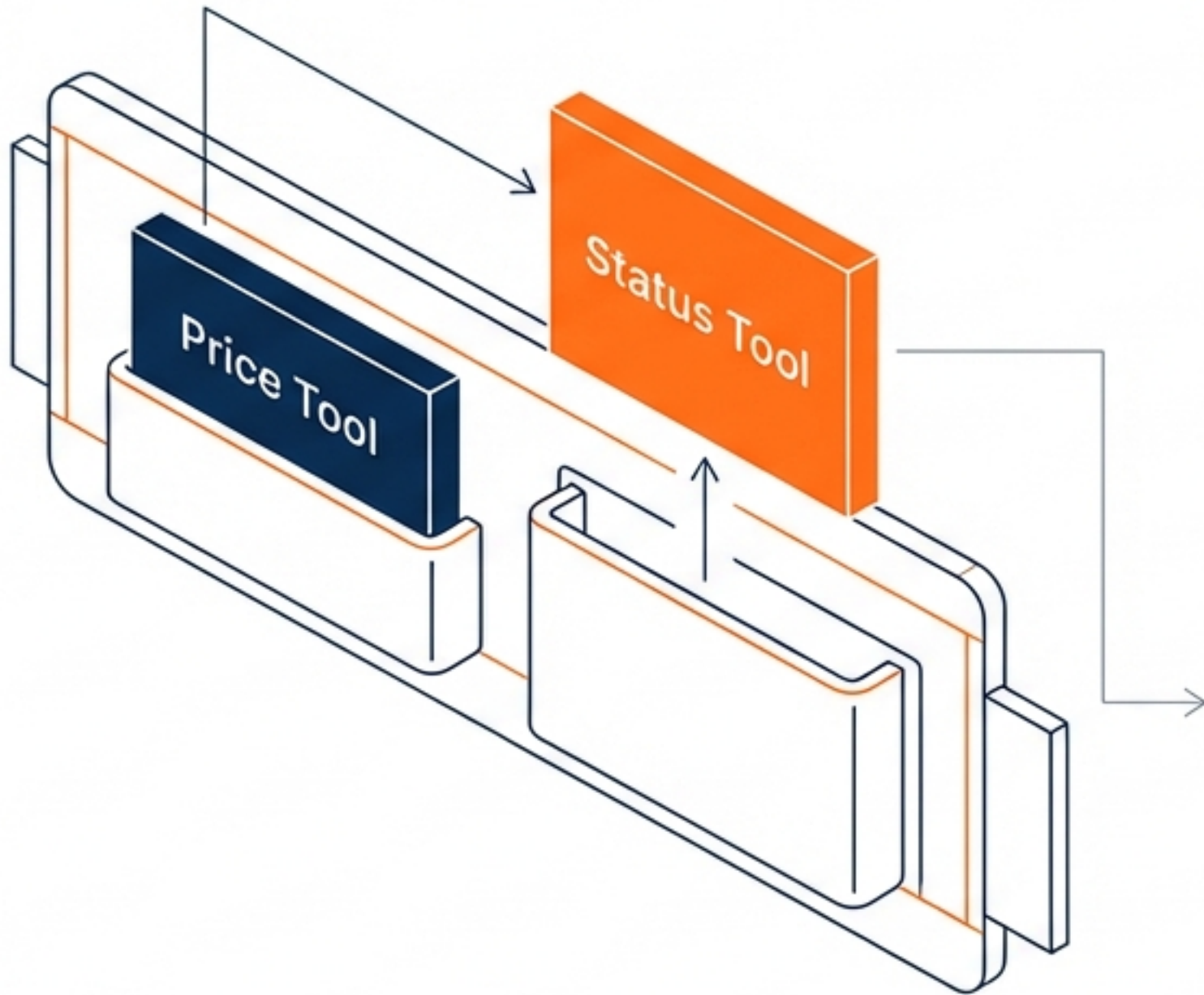
Requisito de Seguridad: Si el vuelo no existe, el sistema no debe fallar. Debe devolver 'Status Unknown'.

Paso 3: Enseñando a la IA (Esquema JSON)

El modelo de lenguaje no 'lee' Python directamente. Necesita una definición en JSON que explique qué hace la herramienta.



Paso 4: Registro de Herramientas

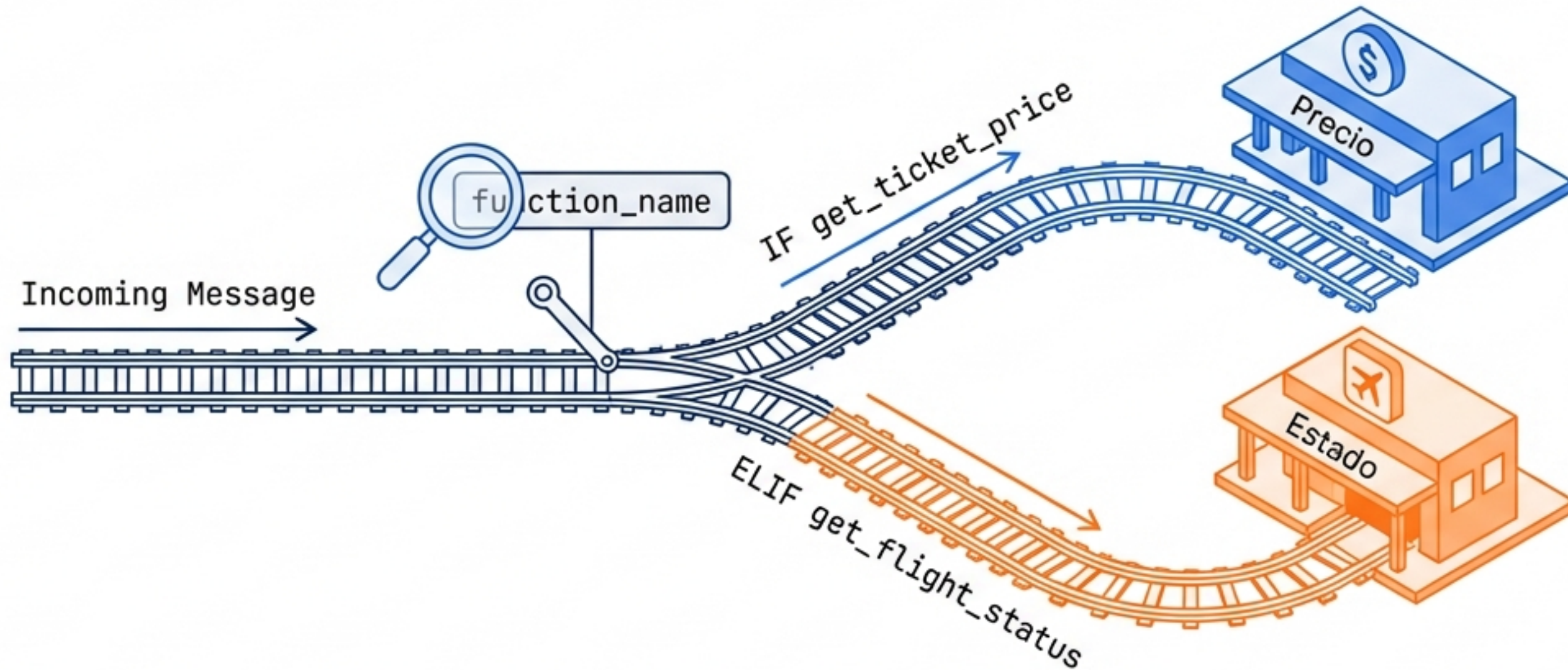


Actualizamos la lista maestra 'tools'. Ahora el asistente tiene acceso a dos capacidades distintas.

Sin este paso, el modelo sabría cómo funciona la herramienta, pero no tendría permiso para usarla.

```
tools = [  
    {'type': 'function', 'function': price_function},  
    {'type': 'function', 'function': status_function} # Nueva integración  
]
```


Paso 5: El Cerebro (Lógica de Enrutamiento)



Este es el paso más crítico. Ya no podemos asumir que el usuario siempre quiere precios.

Estructura de Control:

- **IF** la IA pide `get_ticket_price` -> Ejecutar lógica de precios.
- **ELIF** la IA pide `get_flight_status` -> Ejecutar nueva lógica de estado.

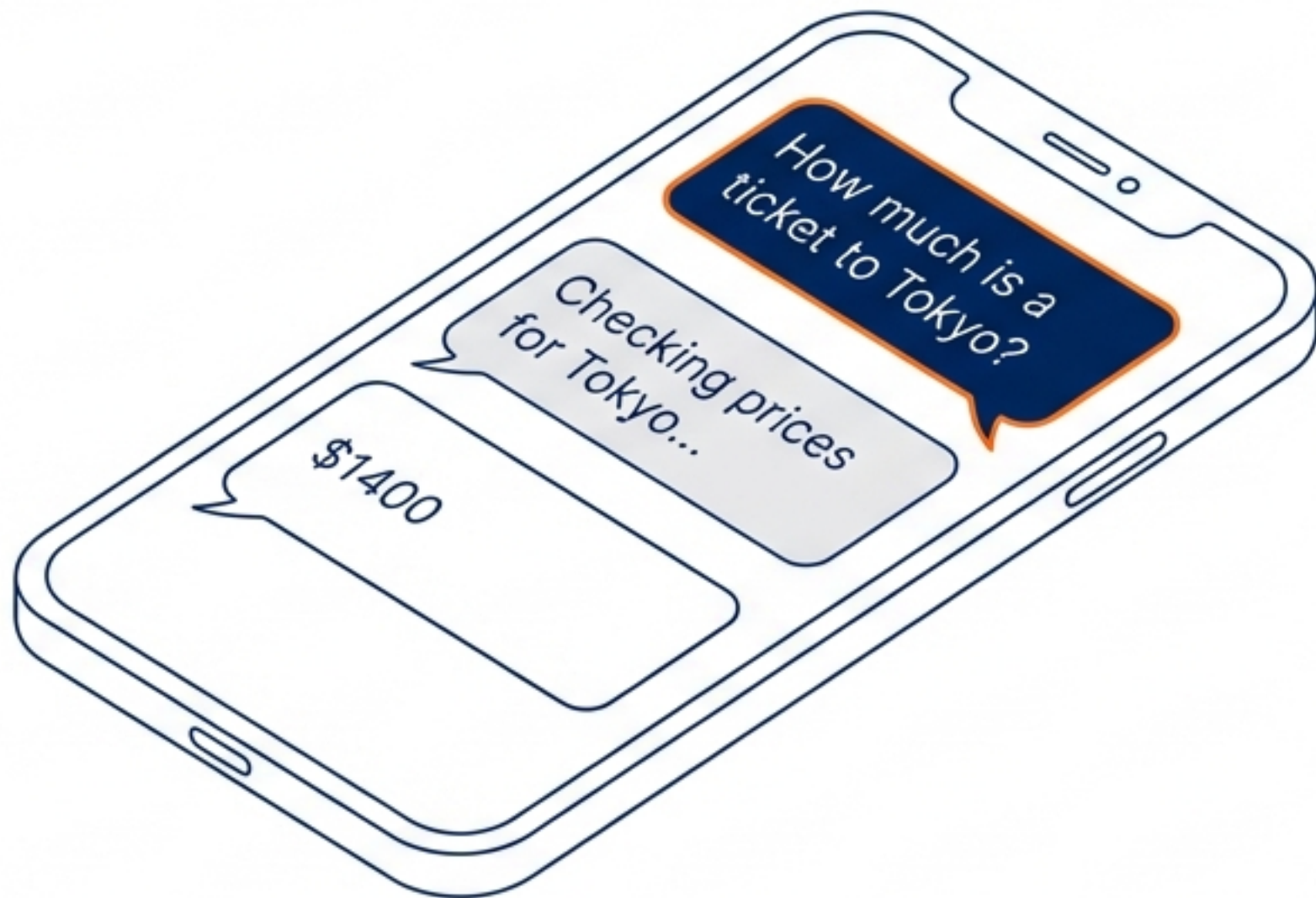
Implementación del Enrutador

```
# 1. Si pide PRECIOS
if function_name == 'get_ticket_price':
    city = arguments.get('destination_city')
    price = get_ticket_price(city)
    price = get_ticket_price(city)
    response_content = {'destination_city': city, 'price':
price}

# 2. Si pide ESTADO DE VUELO
elif function_name == 'get_flight_status':
    flight_number = arguments.get('flight_number')
    status = get_flight_status(flight_number)
    response_content = {'flight_number': flight_number,
'status': status}
```

- Extracción:
Identificamos la función y extraemos los argumentos.
- Lógica Nueva:
Ejecutamos `get_flight_status` y empaquetamos la respuesta JSON.

Validación 1: Prueba de Regresión



Lista de Verificación Técnica

- **Objetivo:** Verificar que lo viejo no se ha roto.
- **Check Interno:** Router identifica 'get_ticket_price'.
- **Resultado Esperado:** \$1400.
- **Estado:** ☒ PASSED

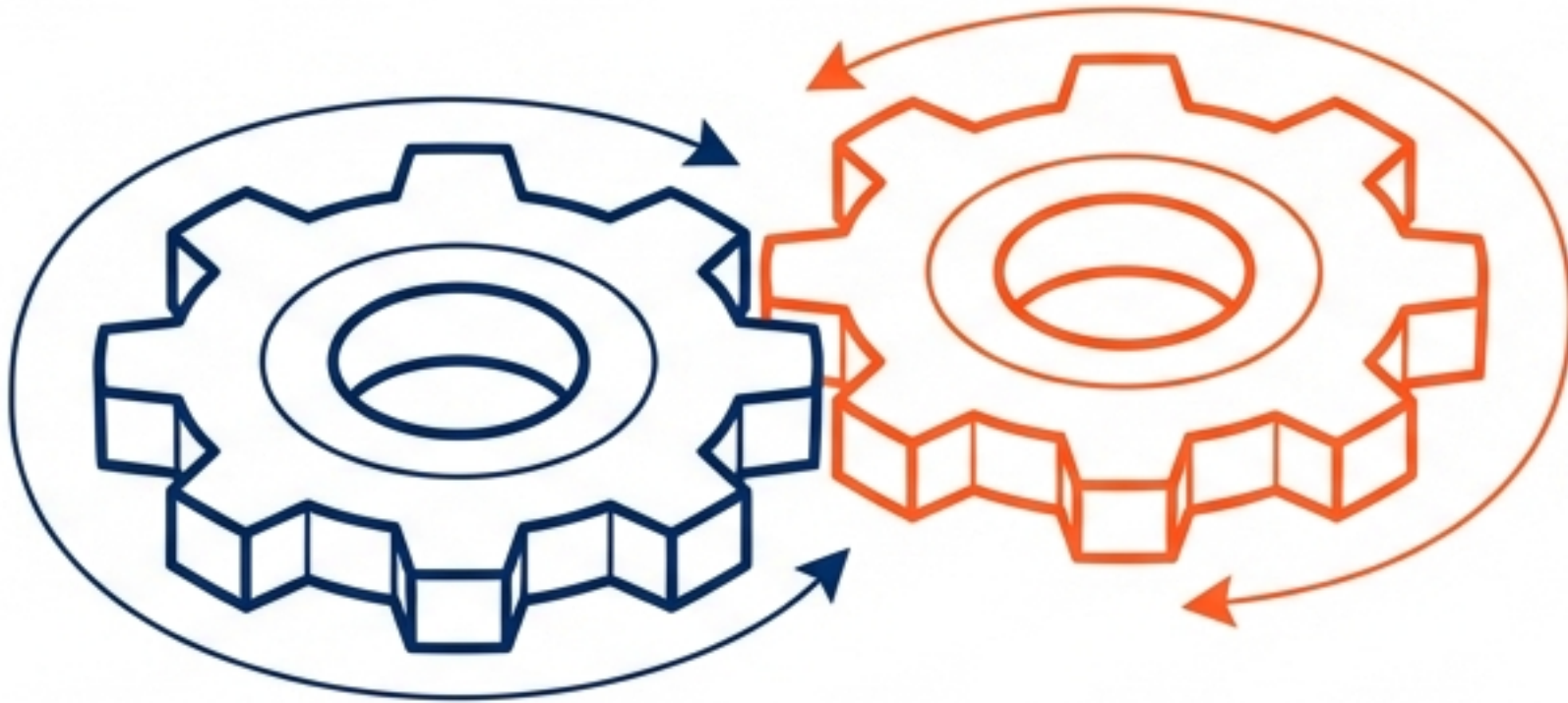
Validación 2: Nueva Funcionalidad



Lista de Verificación Técnica

- **Objetivo:** Probar la misión específica.
- **Check Interno:** Router identifica 'get_flight_status'.
- **Consulta DB:** Busca clave 'FA202'.
- **Estado:**  PASSED

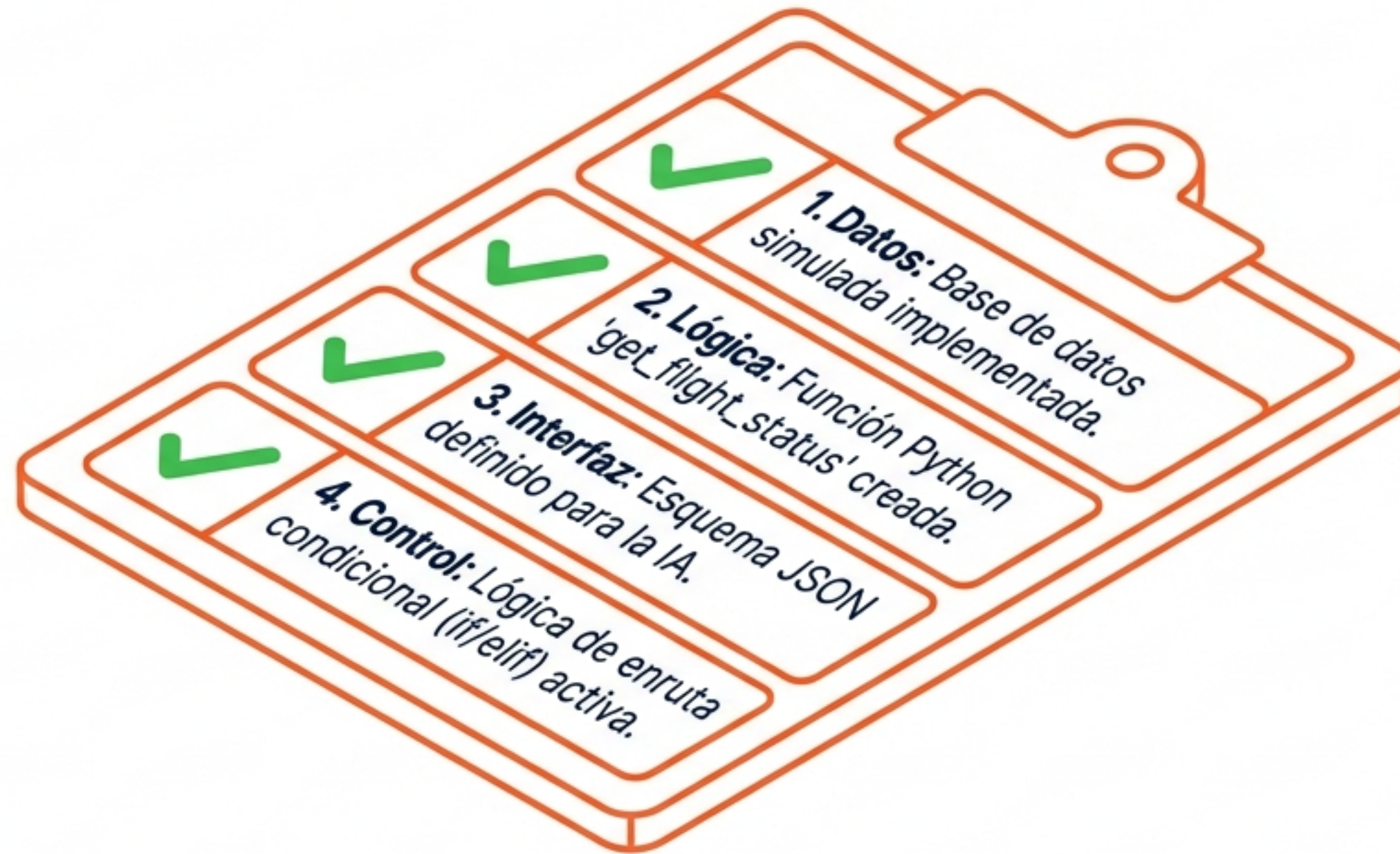
Desafío Avanzado: Consultas Híbridas



User Query: "Check the status of flight FA303 and tell me the price to Berlin."

- **Comportamiento:** Modelos avanzados (GPT-4o) pueden gestionar múltiples llamadas secuenciales.
- **Inferencia:** El modelo detecta dos intenciones distintas en una sola frase.
- **Resultado:** Ejecuta `get_flight_status('FA303')` Y `get_ticket_price('Berlin')`.

Informe de Misión Completada



FlightAI v2.0 está listo para el despliegue. Has transformado una herramienta simple en una arquitectura escalable.